

AI Registration Assistant

Task ID: AI-SS-001 | **Student:** Abhinav Singh | **Student Code:** DAS002971

Technology: Python, NLTK, scikit-learn, Flask, HTML, CSS, JavaScript, JSON

Problem	Students need clear guidance while completing an internship registration process.
Solution	A conversational assistant that answers FAQs, validates details, manages dialog state, and saves confirmed registrations.
Architecture	Hybrid machine-learning intent classification plus deterministic registration workflow.

1. Introduction

The AI Registration Assistant is a conversational student-support system designed to guide users through internship registration. A chatbot reduces confusion by collecting information step by step, answering common questions, validating entries, and providing immediate confirmation.

2. Objectives

The project implements NLP preprocessing, intent recognition, entity extraction, dialog management, validation, local storage, and web integration. Bonus features include FAQ handling, basic English and Hindi-Hinglish support, sentiment analysis, conversation logging, an admin dashboard, and automated testing.

3. NLP and Machine-Learning Method

Training phrases and labels are stored in intents.json. NLTK Snowball stemming normalizes words. TF-IDF converts words and bigrams into numerical features, while Logistic Regression predicts the most likely intent and a confidence score. Exact normalized pattern matching improves reliability for known FAQ phrases.

4. Dialog Management and Entity Extraction

A deterministic state machine manages the registration stages: name, email, program, year, experience, confirmation, and completion. This flow takes priority over general intent classification while registration is active. Regular expressions extract names and email addresses, and validation functions reject malformed values.

5. Web Application and Storage

Flask provides the chatbot page, chat API, reset API, analytics API, health endpoint, and admin dashboard. Each browser receives an independent conversation session. Successful registrations and conversation logs are stored in JSON files, satisfying the no-database requirement. The frontend uses responsive HTML, CSS, and JavaScript with quick actions and a progress bar.

6. Implemented Features

Core/Bonus Feature	Implementation
Greeting and FAQs	ML intent classifier with trained examples and fallback handling
Information collection	State-based questions for name, email, program, year, and experience
Entity extraction	Regex extraction for names, email addresses, and study year
Validation	Name, email, program length, year, experience, and confirmation checks
Confirmation	Unique registration ID and JSON record
Multilingual support	Basic Hindi/Hinglish registration commands and responses
Sentiment and logging	Simple lexicon sentiment plus JSON chat logs
Admin dashboard	Registration totals, messages, intent distribution, sentiment, and recent records
Testing	Seven pytest tests covering NLP, validation, complete flow, and Flask routes

7. Testing and Results

Automated tests verify greeting classification, entity extraction, invalid-email handling, FAQ response, the complete registration workflow, the health endpoint, and the chat API. All seven tests pass. Manual checks also cover cancellation, restarting, unknown questions, responsive layout, confirmation, JSON records, and analytics.

8. Limitations and Future Scope

The intent model uses a small custom dataset and is intended for an internship demonstration rather than production use. Future improvements could include a larger multilingual dataset, spaCy named-entity recognition, transformer models, encrypted storage, admin authentication, email verification, deployment, and a production database.

9. Conclusion

The project meets the required objectives by combining NLP, machine learning, entity extraction, validation, dialog management, registration confirmation, local storage, and web integration. The hybrid design is easy to explain, reliable for structured registration, and straightforward to extend.